

1. Missing Context

A model can fail because it reasons badly, but many agent failures begin earlier: the evidence needed to choose a correct action is absent, hidden, or replaced by an assumption. In repository work, context can include code, tests, build commands, issue history, design references, runtime state, screenshots, environment variables, external documentation, and prior failed attempts.

The naive response is to paste everything into the prompt. That creates a second problem: relevant facts become harder to distinguish from stale or contradictory material. Context quality is not context volume.

2. Three Sources of Context

Direct context is supplied by the user: the goal, constraints, examples, screenshots, files, and explicit preferences. Discovered context is found by Codex in the repo, tools, or current documentation. Durable context is stored for future work in files such as AGENTS.md, skills, project docs, tests, or configuration.

These sources should not be treated equally. An explicit current instruction outranks a stale convention. A test may reveal actual behavior that contradicts a README. A screenshot can establish visual state but not the cause of the bug. Good agent work tracks what each source is evidence for.

3. The Repository-Evidence Principle

Before proposing a code change, establish at least three things where applicable:

1. where the behavior is implemented;
2. how the project expects it to be changed or tested; and
3. what observable fact will distinguish the fix from the current state.

This is why "search first" is not ceremony. If a request says "fix auth redirect," changing the first route named auth may be less intelligent than tracing the request path from a failing test or browser reproduction. The agent should use repository structure as evidence, not as scenery around a guessed patch.

4. References as Constraints

Kundel reports that Astra can infer more product detail from rough ideas when it has good references and access to relevant work. A reference is useful when it reduces uncertainty about an important decision. A production screenshot can constrain spacing and interaction hierarchy; an API schema can constrain data shape; an existing sibling module can constrain architecture and naming.

A weak reference is merely adjacent. "Make it like Apple" leaves almost every decision open. A strong reference says what relation matters: use this existing table's sorting behavior; match this repository's error-object shape; preserve the interaction rhythm in these three recorded flows.

5. When to Ask

OpenAI's Astra guidance notes stronger initiative and a greater tendency to ask when missing input could materially change the result. A useful test is counterfactual: if two reasonable answers to the missing question would lead to substantially different artifacts, ask or surface the assumption. If they would lead to the same next investigative step, investigate first.

Do not ask the user where a function is before searching. Do ask before choosing whether an irreversible migration may discard data. Do not ask what test command exists before inspecting package scripts. Do ask when the desired compatibility floor changes the implementation strategy.

6. The Context Ledger

For difficult work, keep a compact ledger:

Known: observed facts and their sources.

Required: success conditions.

Unknown: questions whose answers may change the approach.

Tried: failed attempts and evidence from them.

Next: the smallest discriminating action.

The ledger prevents repeated exploration and makes a long task reviewable. It also gives Astra useful material for notes across context windows without requiring a narrative transcript of everything that happened.

References: OpenAI, GPT-6 Astra model guidance; OpenAI, GPT-6 Astra launch; Dominik Kundel, "How I Stopped Needing to Explain Things to Codex."

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 3: The Problem of Missing Context

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>